

VLA-ACL: Action-Consistent Visual Token Pruning for Efficient Vision-Language-Action Models

Owen Du^{1,2,*}, Yang Yue^{2,*}, Jie Zhang², Jiaqi Pi², Chi Bene Chen², Gao Huang^{2,†}

Abstract—Vision-Language-Action (VLA) models achieve strong robotic manipulation performance but incur high computational costs from processing long token sequences at every control step, limiting real-time deployment. Visual token pruning offers a direct solution, as visual patches dominate the input sequence and contain considerable redundancy. Existing approaches, however, either rely on indirect training-free heuristics, such as attention scores and motion thresholds, or require costly fine-tuning of the base VLA model. We introduce VLA-ACL (*Action Consistency Learning*), which learns a lightweight visual token pruning policy through action-level supervision while keeping the base VLA model entirely frozen. The training objective encourages actions produced from pruned visual contexts to remain consistent with the full-context teacher, with ground-truth actions as auxiliary supervision. This directly ties token selection to its effect on the downstream control output. Experiments on LIBERO and real-world manipulation tasks show that VLA-ACL prunes up to 87.5% of visual tokens while retaining competitive performance, reduces computation by up to 75%, and achieves a $1.5\times$ inference speedup. These results establish a stronger performance–efficiency trade-off than existing frozen-VLA pruning methods and demonstrate the value of action-level supervision for visual token selection. Code [available](#).

I. INTRODUCTION

Vision-Language-Action (VLA) models combine pre-trained vision-language representations with action prediction to achieve strong performance in robotic manipulation [1–4]. However, their computational cost limits real-time deployment. At every control step, these models process long multimodal sequences dominated by visual tokens from camera observations. Processing these tokens through a large language backbone incurs substantial computation and latency, constraining the frequency at which the policy can respond to changes in the environment.

One promising direction for closing this gap is to exploit visual redundancy through token pruning or caching. Pruning reduces the number of visual tokens processed, while caching reuses visual representations across control steps. Prior work [5] shows that VLA policies can tolerate even random token removal at moderate pruning rates, suggesting substantial redundancy in the visual input. The challenge is to exploit this redundancy more aggressively while preserving the visual information needed for effective control.

Existing approaches largely fall into two categories. Training-free methods [5–12] freeze the base VLA and select tokens using heuristics such as attention scores, temporal changes, and motion thresholds. However, these signals only

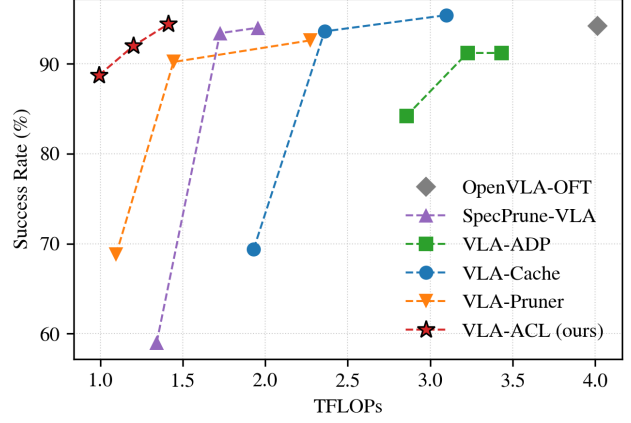


Fig. 1: LIBERO-Long success rates vs. computational budget. VLA-ACL achieves a better performance–efficiency trade-off than existing frozen-VLA pruning and caching methods.

indirectly estimate token importance and do not capture the actual impact of token removal on model performance. Moreover, such methods often require carefully tuned layer- or timestep-specific hyperparameters or access to internal states and attention maps, limiting compatibility with existing acceleration techniques. Training-based approaches [13, 14] instead learn token selection jointly with the base VLA, incurring substantial backbone fine-tuning costs [2]. This also couples the resulting VLA checkpoint to the specific pruning mechanism, limiting its deployment flexibility. These limitations motivate a central question: *Can we learn effective visual token selection from its impact on downstream actions while keeping the base VLA frozen?*

To this end, we introduce VLA-ACL (*Action Consistency Learning*), a visual pruning framework for VLAs that trains a lightweight token selection policy based on its effect on the predicted action: actions produced from pruned visual contexts should remain **consistent** with those produced by the same VLA using the full visual context (Fig. 2). Combined with auxiliary ground-truth action supervision, this teacher-consistency objective directly links visual token selection to downstream control behavior. VLA-ACL trains a patch scoring network to assign an importance score to each visual token and retains the K highest-scoring tokens per camera view before passing them to the language backbone at inference. During training, to enable gradient flow to the patch scorer, we use a soft Top- K relaxation [15] to gate visual embeddings and gradually sharpen the selection weights, reducing the mismatch with hard pruning at inference. Our framework neither accesses nor modifies

¹ETH Zurich ²Tsinghua University

*Equal contribution

†Corresponding author

the internal states of the base model, making it broadly compatible with existing VLA architectures.

Empirically, VLA-ACL requires just a few hours of training on a single GPU, using only a lightweight Transformer [16] as the patch scorer, and avoids costly multi-GPU fine-tuning of the full policy. We evaluate VLA-ACL on LIBERO and real-world manipulation tasks. Our framework supports aggressive pruning rates up to 87.5% of visual tokens and reduces FLOPs by up to 75%, while maintaining success rates comparable to full-context inference. These results demonstrate a strong performance–efficiency trade-off against frozen-VLA pruning baselines, as shown in Fig. 1 and Table I. VLA-ACL thus establishes action consistency as an effective learning signal for visual token selection, enabling aggressive pruning while preserving the information needed for control.

II. RELATED WORK

A. Vision-Language-Action Models

Vision-Language-Action (VLA) models build on pre-trained vision-language representations [17, 18] to predict robot actions from visual observations and language instructions [19, 20]. Representative models include OpenVLA [1] and the π -series [3, 4]. OpenVLA processes visual patch tokens and language instructions through a large language backbone [21], while OpenVLA-OFT [2] improves action-generation efficiency through parallel decoding of action chunks. Despite these advances, processing long visual token sequences remains computationally expensive, hindering their practicality for real-world tasks.

B. Efficient VLA Inference

VLA inference acceleration has been explored through compact architectures [22, 23], dynamic depth control [24], and visual token pruning. Among pruning approaches, *training-free* methods operate on a frozen VLA stack and offer plug-and-play inference acceleration. Building on token pruning for vision-language models [25–27], these methods commonly score visual tokens using text-to-vision attention magnitudes. Recent approaches further incorporate feature diversity, edge saliency, temporal feature changes, and moving averages of action-to-vision attention to guide token selection in sequential robotic observations [5, 9–11].

Besides selection criteria, methods further differ in layer-aware treatment, as attention patterns shift across the language backbone’s layers. Attention scores from deeper layers, which more directly reflect action-relevance, are used to guide token selection [8, 9]. However, since these scores are only available from the previous forward pass, relying on them degrades performance at low retention rates [28]. Other methods instead vary the pruning rate itself across layers, guided by heuristics such as attention entropy [5, 6, 10].

Pruning is further regulated along the temporal dimension: full-context forward passes are executed at the beginning of the control sequence to fill up caches [6, 9] or the pruning rate is adjusted according to signals such as end-effector

velocity [5, 7, 11]. In both cases, these strategies rely on carefully tuned thresholds.

These layer and time-varying mechanisms come with an additional cost: accessing and modifying internal attention mechanisms are incompatible with fused-attention kernels such as FlashAttention [29] or graph-compiled inference, offsetting the very speedups they aim to provide. Taken together, training-free methods still struggle at high pruning rates, as they rely on a growing stack of heuristic-driven strategies that each require carefully tuned hyperparameters, rather than a signal tied directly to task performance.

Training-based methods instead learn the pruning decision jointly with the VLA. To handle the non-differentiable hard pruning objective, LightVLA [13] trains the entire VLA stack using a Gumbel-softmax [30] token selector. Grid-S [14] replaces discrete token selection with a differentiable grid-sampling module fine-tuned end-to-end. Both report strong performance at aggressive pruning ratios, but the pruner and the backbone have to be re-trained, which can take multiple days even on 8 frontier GPUs [2] using parameter-efficient fine-tuning methods such as LoRA [31].

VLA-ACL sits in the gap between these two regimes: like the training-free methods, it keeps the VLA frozen, and it trains only a small patch scorer in just a few GPU hours; like the training-based methods, the pruning decisions are learned. A single hyperparameter K controls how many visual tokens are kept per view, applied before tokens ever enter the language backbone. This mechanism remains constant at every control step, eliminating the need for temporal controllers and keeping VLA-ACL compatible with existing inference-acceleration techniques. Crucially, rather than scoring tokens through intermediate proxies, VLA-ACL learns this scoring function by optimizing action consistency between pruned and full-context forwards of the same frozen VLA, directly tying token selection to its effect on the predicted action.

III. METHOD

VLA-ACL learns a lightweight visual token pruning policy through action-level supervision while keeping the original VLA model frozen. A small Transformer-based [16] patch scorer assigns an importance score to each visual token. At inference, the K highest-scoring tokens are retained before entering the language backbone. During training, the scorer is optimized to maintain action consistency between pruned-context predictions and full-context teacher predictions, with ground-truth actions providing auxiliary supervision. A differentiable soft pruning module combines soft Top- K [15] with a custom gating mechanism to enable gradient propagation to the scorer. Fig. 2 illustrates the training (a) and inference (b) pipelines.

Formally, let $\mathcal{E}$ denote the frozen vision encoder with embedding dimension d . Given a single-view image I , the encoder produces N_e vision embeddings $\mathbf{X} = \mathcal{E}(I) \in \mathbb{R}^{N_e \times d}$. The trainable patch scorer $\mathcal{P}$ assigns each token a logit, yielding $\mathbf{l} = \mathcal{P}(\mathbf{X}) \in \mathbb{R}^{N_e}$. For multi-view inputs (e.g. wrist cameras), $\mathcal{P}$ jointly processes the concatenated

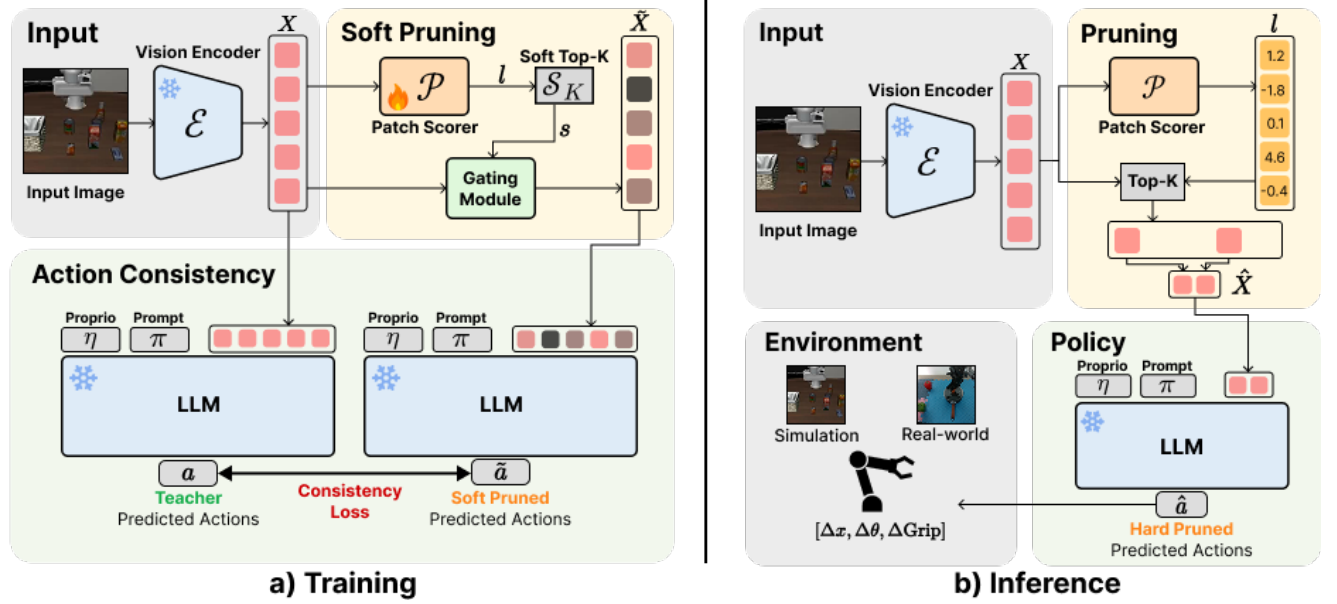


Fig. 2: Overview of VLA-ACL. Modules with a snowflake symbol are kept frozen, modules with a flame symbol receive gradient updates. **a)** During training, two forward passes of the frozen LLM are performed. On the left, the full-vision-context teacher action is generated. On the right, vision embeddings enter our differentiable soft pruning gate and produce the soft-pruned action. **b)** During inference, the hard pruning gate only lets through K patches per view with the highest patch scores, obtained by the trained patch scorer. The pruned tokens never enter the VLA language backbone.

embeddings from all views, while pruning and gating are applied independently to each view with a per-view token budget of K .

A. Token-Pruning: Inference

At each control step, VLA-ACL retains the K visual tokens with the highest patch scores l . Let $i_1, \dots, i_{N_e}$ index the tokens in descending score order:

$$l_{i_1} \geq l_{i_2} \geq \dots \geq l_{i_{N_e}}, \quad (1)$$

$$\text{Top-}K(\mathbf{X}; l) = [\mathbf{X}_{i_1}, \mathbf{X}_{i_2}, \dots, \mathbf{X}_{i_K}]. \quad (2)$$

The selected embeddings pass through the frozen language backbone to predict actions conditioned on proprioceptive inputs η and language instructions π :

$$l = \mathcal{P}(\mathbf{X}) \in \mathbb{R}^{N_e}, \quad (3)$$

$$\hat{\mathbf{X}} = \text{Top-}K(\mathbf{X}; l) \in \mathbb{R}^{K \times d}, \quad (4)$$

$$\hat{\mathbf{a}} = \text{LLM}(\hat{\mathbf{X}}, \eta, \pi). \quad (5)$$

Discarded tokens are excluded from this computation. Each retained token preserves its original RoPE position index [32], maintaining the positional relationships of the full visual sequence.

B. Action Consistency Learning

Action Consistency Learning trains the patch scorer to preserve the frozen VLA’s action predictions under visual token pruning. The full-context VLA provides teacher supervision, complemented by ground-truth actions. However, hard Top- K selection is piecewise constant with respect to the patch logits and has zero gradients almost everywhere. We therefore introduce a differentiable training surrogate that assigns soft selection scores and gates visual embeddings,

allowing action-level supervision to reach the scorer through the frozen VLA.

Soft token selection. We use soft Top- K , a differentiable relaxation [15] that maps patch logits l to continuous saliency scores. Formally, the operator $\mathcal{S}_K$ determines an adaptive threshold θ_K such that the total selection mass equals K , assigning each token a score based on its logit’s scaled distance from this threshold:

$$\mathbf{s} = \mathcal{S}_K(l) \in [0, 1]^{N_e}, \quad (6)$$

$$s_i = \text{Lap}\left(\frac{l_i - \theta_K}{\alpha}\right), \quad \sum_{i=1}^{N_e} s_i = K. \quad (7)$$

where Lap denotes the standard Laplace cumulative distribution function and $\alpha > 0$ controls selection softness. As $\alpha \rightarrow 0^+$, $\mathcal{S}_K$ approaches exact binary Top- K . We anneal α with a cosine schedule to progressively sharpen selection while keeping the budget K fixed.

Differentiable gating. We use the selection scores $\mathbf{s}$ from Eq. 7 to construct soft-pruned visual embeddings $\tilde{\mathbf{X}}$ without discretely removing tokens. Let I_{black} denote a black image processed with the same input normalization as the original observations. The frozen vision encoder produces black image embeddings $\mathbf{b}$, which are combined with the original embeddings through a differentiable gate:

$$\mathbf{b} = \mathcal{E}(I_{\text{black}}) \in \mathbb{R}^{N_e \times d}, \quad (8)$$

$$\tilde{\mathbf{X}}_i = \mathbf{X}_i * s_i + \mathbf{b}_i * \sqrt{1 - s_i^2}. \quad (9)$$

The saliency score s_i determines how much of the original vision embedding $\mathbf{X}_i$ ’s information survives: for s_i close to 1, $\mathbf{X}_i$ is fully preserved, while $s_i \rightarrow 0$ replaces the patch entirely with the black patch embedding. Using the encoder’s

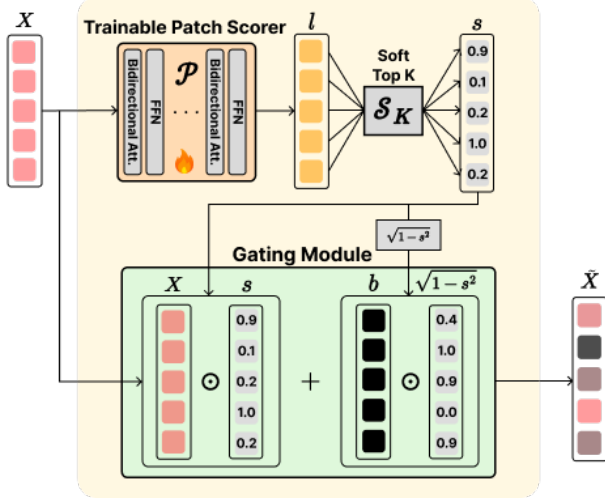


Fig. 3: The soft pruning gate consists of the trainable patch scorer, which outputs a logit for each vision patch. The logits pass through a soft Top- K function, which maps logits to $[0, 1]$ scores summing to K . These scores regulate the information content per patch inside the gating module, where patches with low scores are perturbed using embeddings b of a pitch-black image.

representation of a black image keeps the replacement in-distribution for the frozen backbone, unlike zero vectors or random noise. The resulting soft-pruned embeddings $\tilde{X}$ serve as a differentiable surrogate for the hard-pruned $\hat{X}$, allowing action-level gradients to reach the patch scorer $\mathcal{P}$. The soft pruning gate is shown in Fig. 3.

Action-level supervision. Both original X and soft-pruned vision embeddings $\tilde{X}$ pass separately through the frozen LLM, together with proprioception data η and language instructions π :

$$\tilde{a} = \text{LLM}(\tilde{X}, \eta, \pi), \quad (10)$$

$$a = \text{LLM}(X, \eta, \pi). \quad (11)$$

With the obtained teacher action a and soft-pruned action $\tilde{a}$, we optimize the action consistency loss $\mathcal{L}_{\text{acl}}$ with auxiliary ground-truth supervision a_{gt} :

$$\mathcal{L}_{\text{acl}} = \lambda \|\tilde{a} - a\|_1 + \mu \|\tilde{a} - a_{\text{gt}}\|_1, \quad (12)$$

where λ and μ weight the consistency and ground-truth terms, respectively. The consistency term encourages token selection to preserve the full-context teacher’s actions, while the auxiliary term anchors predictions to demonstrated actions when the teacher is inaccurate. Section IV-D provides more detailed analysis and ablations. Gradients of $\mathcal{L}_{\text{acl}}$ propagate through the frozen VLA and differentiable gate to update only the patch scorer.

We optimize this objective with a fixed budget K , analogous to Top- K sparse autoencoders [33], while annealing progressively sharpens selection. Unlike an ℓ_1 sparsity penalty, the fixed budget directly controls the inference pruning ratio without an additional regularization hyperparameter.

IV. EXPERIMENTS

We evaluate VLA-ACL on the LIBERO [34] simulation benchmark and on real-world manipulation tasks. Furthermore, we conduct ablation studies examining our training objective, token selection strategy, and gating mechanism. We compare against other frozen-VLA pruning and caching methods built on OpenVLA-OFT. To test generality beyond this backbone, we additionally apply VLA-ACL to $\pi_{0.5}$ (Appendix B). All experiments and measurements are conducted using a single NVIDIA A100 GPU.

A. Training

The patch scoring network is a 5-layer bidirectional Transformer that operates on the vision embeddings produced by the vision encoder for each camera view. For the LIBERO suite, we use two camera views, each represented by 256 visual tokens. For two real-world tasks, we use three camera views: two wrist cameras and one static camera. For each task, we train and evaluate the model at $K = 32$ and $K = 64$, corresponding to a token pruning rate of 87.5% and 75%, respectively. These values span an aggressive and a moderate pruning regime, trading off performance against efficiency.

We use the following training hyperparameters uniformly across all LIBERO and real-world tasks, and for both VLA backbones (Appendix A). We set $\lambda = 0.5$ and $\mu = 0.5$, equally weighting action consistency to the teacher and to the ground-truth demonstration. α is annealed from 2.0 to 0.1 across 5000 training steps, closing the gap between soft and hard pruning regimes over the course of training. We optimize with AdamW [35], using a learning rate of 1×10^{-4} and a batch size of 8.

B. Simulation Experiments

Experiment setup. We evaluate on LIBERO’s four suites (Spatial, Object, Goal, Long), testing spatial reasoning, object recognition, goal-directed behavior and long-horizon planning. We adopt the LIBERO evaluation settings of OpenVLA-OFT, consistent with other baselines, where each control step predicts a chunk of eight consecutive actions. Success rates of VLA-ACL are averaged across three random seeds with 50 trials per task, yielding 1500 episodes per task suite. Baseline success rates are taken directly from the corresponding publications. We restrict comparisons to methods that keep the underlying VLA frozen, matching VLA-ACL’s backbone setting and enabling a controlled comparison of pruning strategies. Latency and FLOPs of all methods are computed following the protocol of VLA-Cache [6], measuring the TFLOPs and inference time of the LLM. These efficiency metrics are widely adopted across VLA acceleration methods.

Experiment results. Table I summarizes results on the LIBERO benchmark. At $K = 32$, VLA-ACL reduces FLOPs by 75% and achieves a $1.5\times$ speed-up relative to the OpenVLA-OFT baseline, at the cost of only a marginal decrease (-1.4%) in average success rate. At $K = 64$, our method reduces FLOPs by 65% and achieves a $1.4\times$ speed-up, while slightly exceeding the baseline’s average ($+0.2\%$).

TABLE I: Results of **VLA-ACL** on the LIBERO suite compared with OpenVLA-OFT and other frozen-VLA pruning and caching methods.

Method	Success Rate $\uparrow$					TFLOPs	Latency (ms)
	Spatial	Object	Goal	Long	Average		
OpenVLA-OFT	97.8	97.6	97.6	94.2	96.8	4.013	63.49
+ VLA-Cache	98.3	97.5	98.3	95.4	97.4	3.097	66.79
+ VLA-ADP	99.4	98.0	96.4	91.2	96.3	3.139	58.26
+ SAFE-Pruner	98.0	98.0	96.2	93.4	96.4	1.946	56.61
+ SpecPrune-VLA	97.4	95.8	97.7	93.4	96.1	1.726	60.36
+ VLA-Pruner	93.5	96.2	95.2	90.2	93.8	1.420	63.11
+ VLA-ACL (K=32)	98.2	98.2	96.6	88.7	95.4	0.991	42.30
+ VLA-ACL (K=64)	98.7	98.4	96.5	94.2	97.0	1.412	46.82

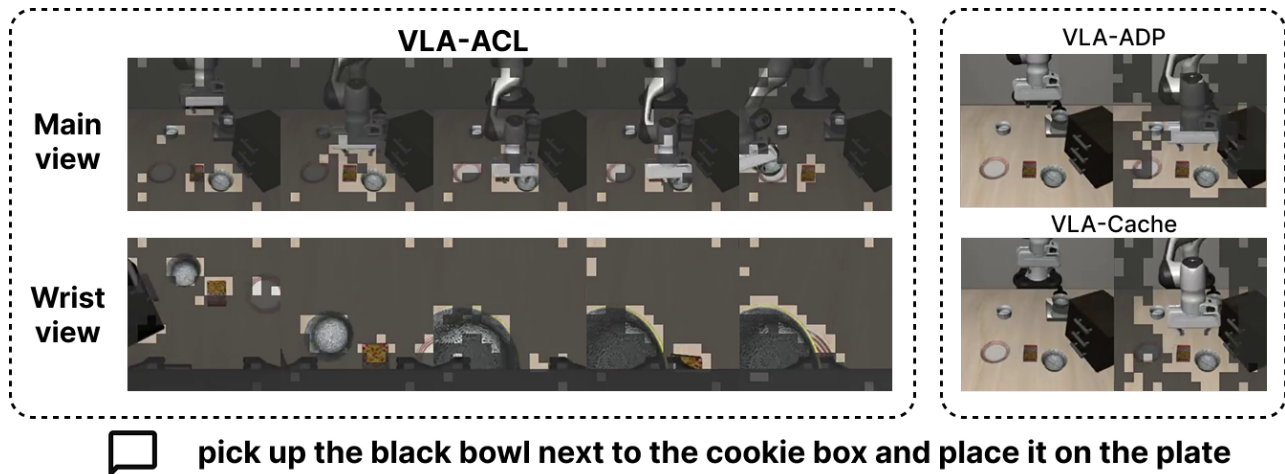


Fig. 4: Visualization of pruning/caching policies on LIBERO-Spatial. Grayed patches are pruned (VLA-ACL, VLA-ADP) or used as previously cached representations (VLA-Cache). VLA-ADP and VLA-Cache both rely on full-context forwards during inference, while **VLA-ACL** sustains a much more aggressive pruning schedule at every control step, enabling lower latency while retaining similar performance across all LIBERO suites.

The two configurations differ most substantially on LIBERO-Long, where long-horizon trajectories are more sensitive to the reduced visual context available at $K = 32$. Relative to other methods, VLA-ACL maintains strong performance under aggressive pruning and achieves the lowest latency. VLA-Cache, SpecPrune-VLA [5], and VLA-Pruner [9] rely on caching internal activations of the LLM, breaking compatibility with FlashAttention [29] and requiring extra GPU time to allocate a `DynamicCache`. Consequently, latency gains from FLOP savings are offset: despite a reduction in FLOPs, VLA-Cache runs slower than base OpenVLA-OFT across the GPU configurations we tested.

VLA-ACL also transfers across VLA backbones: applied to $\pi_{0.5}$ [4] with identical hyperparameters and a single scorer trained jointly on all four suites, it ($K=64$) reduces TFLOPs by 67% and pre-fill latency by $2.2\times$ at 96.1% average success (-0.8 vs. the unpruned model). It outperforms SAFE-Pruner [8] in success rate ($+0.6$) with less than half of its TFLOPs (Appendix B) while retaining `torch.compile`.

Visualizations. Fig. 4 visualizes VLA-ACL’s pruning policy across an entire LIBERO-Spatial episode, compared to VLA-ADP [7] and VLA-Cache. VLA-ACL prunes every control step (including the first frame) at a constant rate

without temporal controllers or falling back to full context, as opposed to the other methods. Our pruning policy reveals which patches contribute most to the action produced by the frozen VLA, whereas training-free methods prune based on heuristics such as internal attention values that only indirectly reflect the token’s attribution toward the VLA action.

C. Real-world experiments

Experiment setup. We evaluate on an ALOHA-style [36] bimanual teleoperation platform (AgileX PiPER-X). Following OpenVLA-OFT’s real-world experiment protocol, we set the action chunk size to 25, corresponding to one second of execution in real time. Each of the four tasks is trained on 50–100 expert demonstrations. Tasks 1 and 4 evaluate instruction following and precise perception; task 4 is additionally a long-horizon task comprising multiple subtasks (lifting the pot lid, picking the correct vegetable, placing the lid back). For both tasks, we vary object positions and the corresponding language instruction across 4 configurations. Tasks 2 and 3 assess spatial understanding and goal-directed behavior, with task 3 featuring multiple subtasks. Tasks 2 and 3 use three camera views and both arms, while the remaining tasks use two and only the right arm. Fig. 5 shows the main

TABLE II: Results of **VLA-ACL** at $K = 64$ on real-world tasks compared with OpenVLA-OFT.

Method	Success Rate $\uparrow$					TFLOPs	Latency (ms)
	Task 1	Task 2	Task 3	Task 4	Average		
OpenVLA-OFT	75	95	85	40	73.75	6.257	91.5
+ VLA-ACL	72.5	95	80	42.5	72.5	2.953	62.1

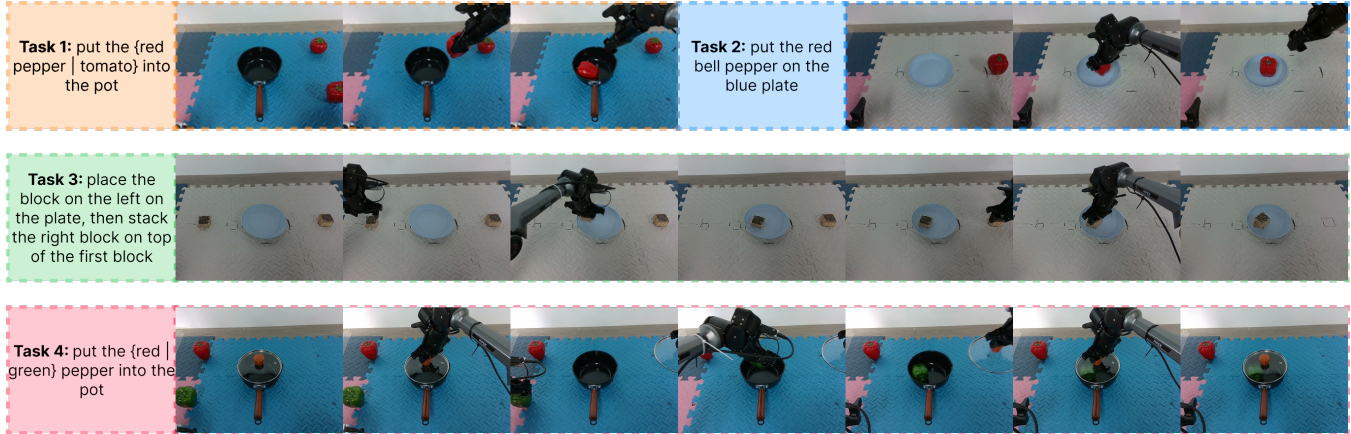


Fig. 5: Real-world experiment task trajectories captured with RealSense D435 cameras.

camera view along each task trajectory and task instructions.

Experiment results. Table II reports results on the real-world tasks, where VLA-ACL is evaluated at $K = 64$. We run 10 trials per configuration for tasks 1 and 4 (40 trials per task in total) and 20 trials for tasks 2 and 3, respectively. VLA-ACL trails the OpenVLA-OFT baseline by a single failed trial on tasks 1 and 3, while matching or exceeding it on the other two tasks. Notably, both models excel at simple pick-place tasks (tasks 2 and 3) but face slight difficulties with fine-grained visual perception in task 1, and significant challenges when combining instruction following with long-horizon execution in task 4. In total, we measure a FLOPs reduction of more than 50% and a speed-up of around $1.5\times$. Averaged across all four tasks, VLA-ACL delivers performance comparable to full-context baselines in real-world settings while enabling substantially faster inference.

D. Ablation study

The following ablation studies are conducted with OpenVLA-OFT.

Action consistency objective. Table III shows that our action consistency objective with auxiliary ground-truth supervision ($\lambda = 0.5, \mu = 0.5$) outperforms both the teacher-only and ground-truth (GT) only variants across K configurations. Because the fine-tuned OpenVLA-OFT models are trained to an action loss below 0.01, teacher and ground-truth actions coincide closely in most frames. However, this gap can nonetheless spike within an episode; thus, including ground-truth actions in the training objective supplies additional supervision signal. To examine this effect directly, Fig. 6 tracks the ℓ_1 distance of the VLA-ACL predicted action to the full-context teacher and to the GT action over a

single training episode. At certain steps, $\text{VLA-ACL} \leftrightarrow \text{GT}$ not only falls below $\text{VLA-ACL} \leftrightarrow \text{teacher}$ – demonstrating the effect of the auxiliary supervision – but also falls below the $\text{teacher} \leftrightarrow \text{GT}$ itself. Despite VLA-ACL only operating on the frozen VLA, this indicates that the learned pruning policy can push the predicted pruned action closer to the ground-truth signal than the unpruned teacher.

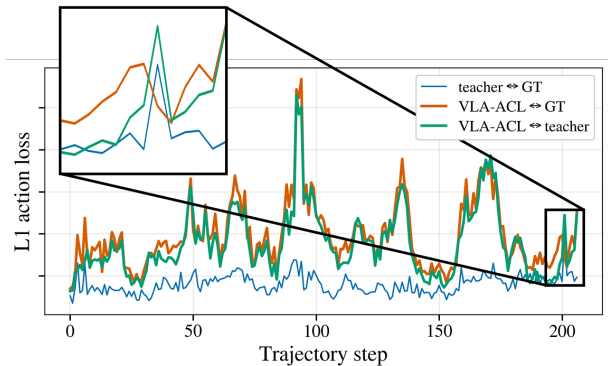


Fig. 6: ℓ_1 distance between VLA-ACL ($K = 64$), teacher, and ground-truth (GT) actions over a LIBERO-Long training episode.

TABLE III: LIBERO success rates of different training objectives.

Teacher	GT	K	Success Rate $\uparrow$
✓	✗	32	95.2
✗	✓		95.1
✓	✓		95.4
✓	✗	64	96.4
✗	✓		95.9
✓	✓		97.0

Selection method. To validate that our action-consistency-based selection outperforms the heuristics used by frozen-

TABLE IV: Comparison of selection methods at equal budget. Action loss denotes $\mathcal{L}_{act}$ with $\lambda = 0.5$ and $\mu = 0.5$. K denotes per-view visual token budget.

Method	K	Pruning layer	Success Rate $\uparrow$ / Action Loss $\downarrow$				
			Spatial	Object	Goal	Long	Average
Attention	32	pre	61.8 / 0.127	55.0 / 0.131	59.0 / 0.096	3.2 / 0.122	44.8 / 0.119
Attention + RoPE		pre	96.6 / 0.061	95.0 / 0.065	96.6 / 0.058	69.6 / 0.064	89.5 / 0.062
VLA-Pruner		3	88.1 / 0.064	87.6 / 0.100	84.9 / 0.070	68.8 / 0.052	82.4 / 0.071
VLA-ACL		pre	98.2 / 0.033	98.2 / 0.043	96.6 / 0.047	88.7 / 0.037	95.4 / 0.040
Attention	64	pre	77.4 / 0.110	59.8 / 0.111	76.6 / 0.079	33.2 / 0.096	61.8 / 0.099
Attention + RoPE		pre	97.4 / 0.028	98.4 / 0.031	96.0 / 0.036	90.2 / 0.033	95.5 / 0.032
VLA-Pruner		3	93.5 / 0.054	96.2 / 0.079	95.2 / 0.063	90.2 / 0.040	93.8 / 0.059
VLA-ACL		pre	98.7 / 0.016	98.4 / 0.022	96.5 / 0.034	94.2 / 0.019	97.0 / 0.023

VLA pruning methods, we compare against three baselines on LIBERO (Table IV). *Attention* denotes the text-to-vision attention heuristic introduced by FastV [25] and adopted by VLA-ADP [7]: first-layer attention scores are used to prune non-salient vision tokens before the forward pass through the language backbone at every control step. *Attention + RoPE* augments this baseline with RoPE [32] positions taken from each token’s original index rather than its position in the pruned sequence, preserving spatial context for the frozen VLA. *VLA-Pruner* combines layer-3 attention scores with an estimate of action-to-vision attention, pruning vision tokens inside the LLM after the third layer. We report success rate and action consistency loss (Eq. 12) for each method across per-view token budgets K .

The results show that plain text-to-vision attention performs poorly, collapsing in low token retention settings on LIBERO-Long ($\sim 3\%$ SR). Adding RoPE positions from the original sequence recovers strong success rates and lower action loss, even surpassing VLA-Pruner. VLA-ACL nonetheless outperforms all baselines in both action loss and success rate, demonstrating that token selection under action-consistency learning retains the most performance at equal token budgets. Success rate and action loss are broadly correlated across methods, although the relationship saturates on some suites: on LIBERO-Goal at $K = 32$, VLA-ACL attains a lower action loss than *Attention + RoPE* (0.047 vs. 0.058) at equal success rates (96.6% for both), suggesting performance is near ceiling on that suite.

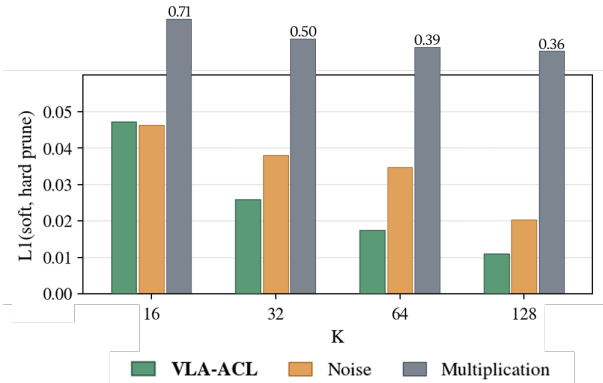


Fig. 7: ℓ_1 between soft-pruned actions of different gating modes vs. hard-pruned actions at different K . Smaller is better.

Gating mode. We show that gating with black-patch embeddings (Eq. 8) effectively restricts information during soft pruning, mimicking the inference-time hard pruning regime. Since the soft-pruned action $\tilde{a}$ should closely track the hard-pruned action $\hat{a}$ — obtained via hard Top- K token selection — to prevent a training-deployment gap, we quantify this via the soft-to-hard ℓ_1 action gap. A smaller difference indicates that soft pruning better emulates hard pruning. We compare VLA-ACL’s gating against noise gating, as used by DiffPrune [37], which substitutes Gaussian noise for black-patch embeddings. Their approach requires training an additional denoiser, since heavily noised patches fall outside the training distribution. Moreover, we compare with multiplicative gating, which simply scales the patches $\mathbf{X}$ directly by their soft Top- K output s and drops the $\sqrt{1 - s^2}$ term in Eq. 9. As shown in Fig. 7 across K values at $\alpha = 0.1$, multiplication maintains a persistently high action loss, while VLA-ACL’s black-patch gating yields a significantly smaller soft-to-hard gap than noise gating for $K \geq 32$. This confirms that our soft pruning scheme closely mirrors the hard Top- K -pruned action used at inference, making it an accurate optimization proxy during training.

V. CONCLUSION

We introduced VLA-ACL, a visual token pruning framework for frozen VLAs that grounds token selection in action consistency: a lightweight patch scorer is trained so that actions produced from a pruned visual context match those of the same model with full context, with ground-truth demonstrations as auxiliary supervision. On LIBERO across two VLA backbones and on real-world tasks, VLA-ACL prunes up to 87.5% of visual tokens and cuts FLOPs by up to 75% while maintaining competitive success rates, at a training cost of only a few GPU hours, extending the Pareto frontier of FLOPs and latency among frozen-VLA visual token pruning methods.

REFERENCES

- [1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “Open-VLA: An open-source vision-language-action model,”

- in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 2679–2713.
- [2] M. J. Kim, C. Finn, and P. Liang, “Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success,” in *Proceedings of Robotics: Science and Systems*, 2025.
 - [3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
 - [4] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. R. Walke, A. Walling, H.-H. Wang, L. Yu, and U. Zhilinsky, “ $\pi_{0.5}$: a vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
 - [5] H. Wang, J. Xu, Y. Xiang, J. Pan, Y. Zhou, Y.-L. Li, and G. Dai, “Specprune-VLA: Accelerating vision-language-action models via action-aware self-speculative pruning,” in *Forty-third International Conference on Machine Learning*, 2026.
 - [6] S. Xu, Y. Wang, C. Xia, D. Zhu, T. Huang, and C. Xu, “VLA-Cache: Efficient vision-language-action manipulation via adaptive token caching,” in *Advances in Neural Information Processing Systems*, vol. 38, Main Conference, 2025, pp. 164 448–164 473.
 - [7] X. Pei, Y. Chen, S. Xu, Y. Wang, Y. Shi, and C. Xu, “Action-aware dynamic pruning for efficient vision-language-action manipulation,” in *International Conference on Learning Representations*, 2026, pp. 10 832–10 851.
 - [8] S. Ma, C. Zhang, C. Wang, Y. Wang, Y. Wu, Z. Wang, J. Tian, Z. Zhu, and Y. Tang, “SAFE-pruner: Semantic attention-guided future-aware token pruning for efficient vision-language-action manipulation,” in *Computer Vision – ECCV 2026*. Springer Nature Switzerland, 2026, pp. 163–180.
 - [9] Z. Liu, Y. Chen, H. Cai, T. Lin, S. Yang, Z. Liu, and B. Zhao, “Bridging the semantic-action gap in visual token pruning for efficient VLA inference,” *arXiv preprint arXiv:2511.16449*, 2025.
 - [10] Y. Yang, Y. Wang, Z. Wen, L. Zhongwei, C. Zou, Z. Zhang, C. Wen, and L. Zhang, “EfficientVLA: Training-free acceleration and compression for vision-language-action models,” in *Advances in Neural Information Processing Systems*, vol. 38, Main Conference, 2025, pp. 40 891–40 914.
 - [11] Y. Li, Y. Meng, Z. Sun, K. Ji, C. Tang, J. Fan, X. Ma, S.-T. Xia, Z. Wang, and W. Zhu, “SP-VLA: A joint model scheduling and token pruning approach for VLA model acceleration,” in *The Fourteenth International Conference on Learning Representations*, 2026.
 - [12] X. Tan, Y. Yang, P. Ye, J. Zheng, B. Bai, X. Wang, J. Hao, and T. Chen, “Think twice, act once: Token-aware compression and action reuse for efficient inference in vision-language-action models,” *arXiv preprint arXiv:2505.21200*, 2025.
 - [13] T. Jiang, X. Jiang, Y. Ma, X. Wen, B. Li, K. Zhan, P. Jia, Y. Liu, S. Sun, and X. Lang, “The better you learn, the smarter you prune: Towards efficient vision-language-action models via differentiable token pruning,” *arXiv preprint arXiv:2509.12594*, 2025.
 - [14] Y. Feng, Z. Zhao, Y. Ma, C. Xia, C. Du, Y. Wang, and C. Xu, “See what matters: Differentiable grid sample pruning for generalizable vision-language-action model,” in *Forty-third International Conference on Machine Learning*, 2026.
 - [15] L. Struski, M. B. Bednarczyk, I. T. Podolak, and J. Tabor, “LapSum - one method to differentiate them all: Ranking, sorting and top-k selection,” in *The International Conference on Machine Learning (ICML) 2025*, 2025.
 - [16] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
 - [17] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2023, pp. 11 941–11 952.
 - [18] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, “DINOv2: Learning robust visual features without supervision,” *Transactions on Machine Learning Research*, 2024.
 - [19] L. Beyer, A. Steiner, A. S. Pinto, A. Kolesnikov, X. Wang, D. M. Salz, M. Neumann, I. M. Alabdulmohsin, M. Tschannen, E. Bugliarello, T. Unterthiner, D. Keysers, S. Koppula, F. Liu, A. Grycner, A. A. Gritsenko, N. Houlsby, M. Kumar, K. Rong, J. M. Eisenschlos, R. Kabra, M. Bauer, M. Bošnjak, X. Chen, M. Minderer, P. Voigtlaender, I. Bica, I. Balazevic, J. Puigcerver, P. Papalampidi, O. Hénaff, X. Xiong, R. Soricut, J. Harmsen, and X.-Q. Zhai, “PaliGemma: A versatile 3b vlm for transfer,” *arXiv preprint arXiv:2407.07726*, 2024.
 - [20] S. Karamcheti, S. Nair, A. Balakrishna, P. Liang, T. Kollar, and D. Sadigh, “Prismatic VLMs: Investigating the design space of visually-conditioned language models,” in *International Conference on Machine Learning*, 2024.
 - [21] H. Touvron, L. Martin, K. R. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava,

- S. Bhosale, D. M. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. S. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. M. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. H. M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [22] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti *et al.*, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [23] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, Y. Peng, F. Feng, and J. Tang, “TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation,” *IEEE Robotics and Automation Letters*, vol. 10, pp. 3988–3995, 2025.
- [24] Y. Yue, Y. Wang, B. Kang, Y. Han, S. Wang, S. Song, J. Feng, and G. Huang, “DeeR-VLA: Dynamic inference of multimodal large language models for efficient robot execution,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 56 619–56 643.
- [25] L. Chen, H. Zhao, T. Liu, S. Bai, J. Lin, C. Zhou, and B. Chang, “An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models,” in *European Conference on Computer Vision*. Springer, 2024, pp. 19–35.
- [26] Y. Zhang, C.-K. Fan, J. Ma, W. Zheng, T. Huang, K. Cheng, D. Gudovskiy, T. Okuno, Y. Nakata, K. Keutzer *et al.*, “SparseVLM: Visual token sparsification for efficient vision-language model inference,” in *International Conference on Machine Learning*, 2025.
- [27] S. R. Alvar, G. Singh, M. Akbari, and Y. Zhang, “DivPrune: Diversity-based visual token pruning for large multimodal models,” in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 9392–9401.
- [28] Q. Luo, S. Liu, H. Zhao, K. Li, X. Wang, N. Su, D. Wang, and Y. Chen, “The gate, not the cache: Gate provenance bounds the closed-loop reliability of training-free VLA token skipping,” *arXiv preprint arXiv:2608.00391*, 2026.
- [29] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with io-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 16 344–16 359.
- [30] E. Jang, S. Gu, and B. Poole, “Categorical reparameterization with Gumbel-softmax,” in *International Conference on Learning Representations*, 2017.
- [31] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [32] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024.
- [33] L. Gao, T. Dupré la Tour, H. Tillman, G. Goh, R. Tross, A. Radford, I. Sutskever, J. Leike, and J. Wu, “Scaling and evaluating sparse autoencoders,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 26 721–26 754.
- [34] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 44 776–44 791.
- [35] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.
- [36] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *Proceedings of Robotics: Science and Systems*, 2023.
- [37] L. He, M. Yao, S. Young, and L. Xu, “Beyond surrogate gradients: Fully differentiable token pruning for vision-language models,” *arXiv preprint arXiv:2605.28051*, 2026.
- [38] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *The Eleventh International Conference on Learning Representations*, 2023.

TABLE V: Results of **VLA-ACL** on the LIBERO suite compared with $\pi_{0.5}$ and other frozen-VLA pruning and caching methods. Success rates and TFLOPs of the baselines are taken from the results reported by SAFE-Pruner [8]; their latency cannot be measured as the code is not public.

Method	Success Rate $\uparrow$					TFLOPs	Latency (ms)
	Spatial	Object	Goal	Long	Average		
$\pi_{0.5}$	98.8	98.2	98.0	92.4	96.9	2.115	25.9
+ VLA-Cache	94.8	96.0	96.4	88.2	93.9	1.632	-
+ VLA-Pruner	95.8	97.2	95.6	88.0	94.2	1.583	-
+ SAFE-Pruner	97.0	98.8	96.0	90.2	95.5	1.482	-
+ VLA-ACL (K=64)	97.4	98.0	96.6	92.3	96.1	0.701	11.6

APPENDIX

A. Implementation Details

OpenVLA-OFT. We use the OpenVLA-OFT model fine-tuned per LIBERO suite, like the methods we compare with, and train a patch scorer for each model on the trajectories of the respective suite. For the real-world experiments, we likewise fine-tune one base policy per task, following the OpenVLA-OFT specification. All ablation studies in Section IV-D are conducted with OpenVLA-OFT.

$\pi_{0.5}$. We further train VLA-ACL on $\pi_{0.5}$, following the same action consistency objective as in Eq. 12. Since $\pi_{0.5}$ produces actions through flow matching [38] rather than direct regression, we adopt the base model’s training procedure for the patch scorer, sampling a noise level per batch and perturbing the ground-truth actions accordingly. The consistency term is then computed between the velocity predicted from the pruned visual context and that of the full-context teacher at the same noise level, while the auxiliary term supervises the predicted velocity with the flow-matching target. In contrast to OpenVLA-OFT, $\pi_{0.5}$ has a single model for all LIBERO suites. We therefore train one patch scorer on top of it jointly on all four suites and evaluate it on each, so the scorer generalizes across suites without per-suite specialization. All training hyperparameters (λ , μ , the α schedule, learning rate, batch size and training steps) are identical to those used for OpenVLA-OFT (Section IV), without any backbone-specific tuning, showing that VLA-ACL generalizes across VLA backbones.

Efficiency measurement. For $\pi_{0.5}$, latency and TFLOPs follow the implementation of SAFE-Pruner [8], where only the LLM pre-fill is measured. The success rates and TFLOPs of VLA-Cache, VLA-Pruner, and SAFE-Pruner are taken from the SAFE-Pruner publication. We cannot measure their latency, as their code for this model is not public. Their temporal and per-layer pruning control, however, prevents the use of `torch.compile`, which we expect to slow down inference. Training and evaluation of all models are performed on a single NVIDIA A100 GPU.

B. More results

Results on the $\pi_{0.5}$ model are summarized in Table V. VLA-ACL at $K = 64$ achieves the best average success rate among the frozen-VLA pruning baselines at the lowest

computational budget, with a 67% reduction in TFLOPs and a $2.2\times$ speedup in pre-fill latency relative to the base model. Compared to the strongest baseline, SAFE-Pruner, VLA-ACL uses less than half of the TFLOPs (0.701 vs. 1.482) while exceeding its average success rate (96.1% vs. 95.5%), trailing the unpruned $\pi_{0.5}$ by only 0.8 points. The gap is concentrated on LIBERO-Object, where SAFE-Pruner is ahead (98.8% vs. 98.0%). Since the baseline implementations are not available to us, we cannot measure their latency directly, but we expect the latency advantage measured on OpenVLA-OFT in Table I to carry over to this backbone. These results further show that a single patch scorer trained jointly across all four suites performs well on each of them.

C. Real-world experiment setup

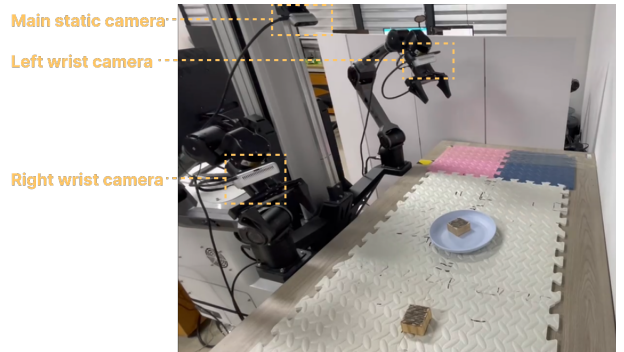


Fig. 8: Our real-world platform: a bimanual AgileX PiPER-X with a main static camera and left and right wrist cameras (RealSense D435).

Fig. 8 shows our bimanual robot platform (AgileX PiPER-X) with one static and two wrist-mounted RealSense D435 cameras. We collect 50–100 expert demonstrations for each of the four real-world tasks using the ALOHA teleoperation platform. Subsequently, we use the trajectory data to fine-tune OpenVLA-OFT using imitation learning.